

• PYTHON •

Python Data Structures

- Tuples -

```
T1 = (1, 2, 3, 4, 5)
T2 = (8, 14.21, "Bangladesh", True)
```

```
print(T1)
print(T2)
```

```
(1, 2, 3, 4, 5)
(8, 14.21, 'Bangladesh', True)
```

```
T1 = (7)          # Integer
T2 = (7, )        # Tuple
```

```
print(type(T1))
print(type(T2))
```

```
<class 'int'>
<class 'tuple'>
```

```
T = (10, 20, 30, 40)
print(T[0])
print(T[-1])
```

```
10
40
```

```
T = ("A", "B", "C")
print(len(T))
```

```
3
```

```
T = (100, 200, 300)
```

```
for Number in T:
    print(Number)
```

```
100
200
300
```

```
T = (10, 20, 30, 40)
```

```
for j in range(len(T)):
    print(j, T[j])
```

```
print("----")
```

```
k = 0
```

```
while k < len(T):
    print(T[k])
    k += 1
```

```
0 10
1 20
2 30
3 40
----
```

```
10
20
30
40
```

Tuple Slicing

Tuple[Start : End : Step]

```
T = (1, 2, 3, 4, 5, 6, 7, 8)
```

Print from starting index to just before the last index

```
print(f"1. {T[:]}")
print(f"2. {T[1:7]}")
print(f"3. {T[1:8:3]}")
print(f"4. {T[:4]}")
print(f"5. {T[4:]}")
print(f"6. {T[::2]}")
print(f"7. {T[::-1]}")
print(f"8. {T[-3:]}")
print(f"9. {T[-4:-1]}")
print(f"10. {T[-5:4]}")
print(f"11. {T[5:-4]}")
print(f"12. {T[-1:-8:-3]}")
print(f"13. {T[1:100]}")
```

1. (1, 2, 3, 4, 5, 6, 7, 8)
2. (2, 3, 4, 5, 6, 7)
3. (2, 5, 8)
4. (1, 2, 3, 4)
5. (5, 6, 7, 8)
6. (1, 3, 5, 7)
7. (8, 7, 6, 5, 4, 3, 2, 1)
8. (6, 7, 8)
9. (5, 6, 7)
10. (4,)
11. ()
12. (8, 5, 2)
13. (2, 3, 4, 5, 6, 7, 8)

```
T1 = (1, 2, 3)
T2 = (4, 5, 6)
```

```
T3 = T1 + T2
print(T3)
```

```
(1, 2, 3, 4, 5, 6)
```

```
T = (10, 25, 50)
```

```
print(min(T))
print(max(T))
```

```
10
50
```

```
T = (505, 506, 507, 508)
```

```
print(sum(T))
```

```
2026
```

```
T = (1, 3, 5, 7, 3, 9, 3, 11)
```

```
print(T.count(3))
```

```
3
```

```
T = (10, 30, 60, 90)
```

```
print(T.index(60))
```

```
2
```

```
T = (6, 2, 4)

print(sorted(T))          # Return as List

print(tuple(sorted(T)))
```

```
[2, 4, 6]
(2, 4, 6)
```

```
# Tuple Packing
Tuple = "KTH", 22, "Student"

print(Tuple)

print("\n")

# Tuple Unpacking
Name, Age, Profession = Tuple

print(Name)
print(Age)
print(Profession)
```

```
('KTH', 22, 'Student')
```

```
KTH
22
Student
```

- Python Dictionaries -

```
# Create Dictionary :      Dict = {"Key1" : "Value1", "Key2" : "Value2", "Key3" : "Value3",...}

Student = {
    "Name": "Tajkir",
    "Age": 22,
    "City": "Chandpur"
}

print(Student)
```

```
{'Name': 'Tajkir', 'Age': 22, 'City': 'Chandpur'}
```

```
Student = dict(Name = "Tajkir", Age = 22, City = "Dhaka")      # Using dict() function
print(Student)
```

```
{'Name': 'Tajkir', 'Age': 22, 'City': 'Dhaka'}
```

```
print(f"1. {Student.keys()}")      # Print keys
print(f"2. {Student.values()}")    # Print values
print(f"3. {Student.items()}")     # Print key-value pairs
```

```
1. dict_keys(['Name', 'Age', 'City'])
2. dict_values(['Tajkir', 22, 'Dhaka'])
3. dict_items([('Name', 'Tajkir'), ('Age', 22), ('City', 'Dhaka')])
```

```
print(Student["Name"])              # Access values by key
print(Student["Age"])
print(Student["City"])
```

```
Tajkir
22
Dhaka
```

```
Student = {"Name": "Tajkir", "Age": 22, "City": "Chandpur"}

print(Student.get("Name"))          # Access values using get() function
```

```
print(Student.get("Age"))
print(Student.get("City"))
```

```
Tajkir
22
Chandpur
```

```
Student["CGPA"] = 3.5          # Add new key-value
print(Student)
```

```
{'Name': 'Tajkir', 'Age': 22, 'City': 'Chandpur', 'CGPA': 3.5}
```

```
Student["Age"] = 23           # Update value
print(Student)
```

```
{'Name': 'Tajkir', 'Age': 23, 'City': 'Chandpur', 'CGPA': 3.5}
```

```
Student = {"Name": "Tajkir", "Age": 22, "City": "Chandpur", "CGPA": 3.5}

Student.update({"Age" : 23, "CGPA": 3.6})      # Using update() function

print(Student)
```

```
{'Name': 'Tajkir', 'Age': 23, 'City': 'Chandpur', 'CGPA': 3.6}
```

```
Student = {"Name": "Tajkir", "Age": 22, "City": "Chandpur", "CGPA": 3.5}

del Student["CGPA"]          # Delete a key

print(Student)
```

```
{'Name': 'Tajkir', 'Age': 22, 'City': 'Chandpur'}
```

```
Student = {"Name": "Tajkir", "Age": 22, "City": "Chandpur"}

Student.pop("Age")           # Using pop() function

print(Student)
```

```
{'Name': 'Tajkir', 'City': 'Chandpur'}
```

```
Student = {"Name": "Tajkir", "Age": 22, "City": "Chandpur"}

Student.popitem()            # Delete last item

print(Student)
```

```
{'Name': 'Tajkir', 'Age': 22}
```

```
Student.clear()              # Delete all items

print(Student)
```

```
{}
```

```
Student = {"Name": "Tajkir", "Age": 22, "City": "Chandpur"}

print(len(Student))
```

```
3
```

```
Student = {"Name": "Tajkir", "Age": 22, "CGPA": 3.5}

if 3.5 in Student.values() :
    print("Value Exists")
else :
    print("Value Does Not Exist")
```

Value Exists

```
Student = {"Name": "Tajkir", "Age": 22}

for key in Student :
    print(key)
```

Name
Age

```
for key, value in Student.items():
    print(key, ":", value)
```

Name : Tajkir
Age : 22

```
Student = {"Name" : "Tajkir", "Age" : 23}

Student_Copy = Student.copy()                # Copying Dictionary

print(Student_Copy)

{'Name': 'Tajkir', 'Age': 23}
```

```
Student = {"Name" : "Tajkir", "Age" : 23}

Student_Copy = dict(Student)                  # Using dict() function

print(Student_Copy)

{'Name': 'Tajkir', 'Age': 23}
```

```
Company = {                                    # Nested Dictionary
    "HR": {"Manager": "A", "Staff": 5},
    "IT": {"Manager": "B", "Staff": 10}
}

print(Company)

{'HR': {'Manager': 'A', 'Staff': 5}, 'IT': {'Manager': 'B', 'Staff': 10}}
```

- Sets -

```
S = {1,2,2,3,4}

print(S)
print(type(S))

{1, 2, 3, 4}
<class 'set'>
```

```
T = set([1, 2, 3])                # Using set() function

print(T)

{1, 2, 3}
```

```
A = set()                                # Set

print(A)
print(type(A))

B = {}                                  # This creates a Dictionary, NOT a Set

print(B)
print(type(B))
```

```
set()
<class 'set'>
{}
<class 'dict'>
```

```
C = {10, 20, 300, 40, 50, 665}

print(C)                                # Unordered Nature

D = {1, 1, 2, 2, 3, 3}

print(D)                                # Only Unique Elementes Will Be Selected

{50, 20, 40, 665, 10, 300}
{1, 2, 3}
```

```
E = {10, 20, 30}

for Item in E :                          # Accessing Elements
    print(Item)
```

```
10
20
30
```

```
Players = {'Mustafiz','Rishad','Tanzid'}

Players.add('Tajkir')                    # Add Single Element
print(Players)
```

```
{'Tanzid', 'Tajkir', 'Mustafiz', 'Rishad'}
```

```
Players = {'Mustafiz','Rishad','Tanzid'}

Players.update(['Emon', 'Shoriful'])     # Add Multiple Elements
print(Players)
```

```
{'Emon', 'Rishad', 'Shoriful', 'Mustafiz', 'Tanzid'}
```

```
Players.remove('Shoriful')               # Remove Specific Element

print(Players)
```

```
{'Emon', 'Rishad', 'Mustafiz', 'Tanzid'}
```

```
Players.discard('Emon')                  # Safe Removal

print(Players)
```

```
{'Rishad', 'Mustafiz', 'Tanzid'}
```

```
Players = {'Mustafiz','Rishad','Tanzid'}

Players.pop()                            # Remove Random Element
print(Players)
```

```
{'Mustafiz', 'Rishad'}
```

```
Players.clear()
```

```
print(Players)
```

```
set()
```

```
# Mathematical Set Operations
```

```
A = {1,2,3,4}
```

```
B = {3,4,5,6}
```

```
print('Union :', A|B)
```

```
print('Intersection :', A & B)
```

```
print('Difference :', A-B)
```

```
print('Symmetric Difference :', A^B)
```

```
Union : {1, 2, 3, 4, 5, 6}
```

```
Intersection : {3, 4}
```

```
Difference : {1, 2}
```

```
Symmetric Difference : {1, 2, 5, 6}
```

```
A = {1, 2}
```

```
B = {1, 2, 3, 4}
```

```
print("Subset:", A.issubset(B))
```

```
# Subset : All Elements of A Are Present Inside B
```

```
print("Superset:", B.issuperset(A))
```

```
# Superset : B Contains All Elements of A
```

```
print("Disjoint:", A.isdisjoint({5, 6}))
```

```
# Disjoint : Two Sets Have No Common Elements
```

```
Subset: True
```

```
Superset: True
```

```
Disjoint: True
```

```
FS = frozenset([1, 2, 3])          # Frozen Set is Immutable
```

```
print(FS)
```

```
# FS.add(5)                        # ❌ Cannot Modify [ERROR]
```

```
frozenset({1, 2, 3})
```

```
# Frozen Set as Dictionary Key
```

```
D = {  
    frozenset([1,2]): "Group A",  
    frozenset([3,4]): "Group B"  
}
```

```
print(D)
```

```
{frozenset({1, 2}): 'Group A', frozenset({3, 4}): 'Group B'}
```

```
List = [1,2,2,3,3,4,5,5]
```

```
Unique = list(set(List))          # Remove Duplicate Elements
```

```
print(Unique)
```

```
[1, 2, 3, 4, 5]
```

```
# Set Comprehension - Syntax : { Expression for Item in Iterable if Condition }
```

```
S = set()
```

```
for i in range(5):
```

```
    S.add(i)                        # Normal Way of Creating Set (Using Loop)
```

```
print(S)
```

```
print(".")
```

```
T = {x for x in range(1, 7)}
```

```

print(T)

print(".")

U = {x for x in range(20) if x % 2 == 0}
print(U)

print(".")

List = [1,2,2,3,3,4,5,5]
Unique = {x for x in List}
print(Unique)

```

```

{0, 1, 2, 3, 4}
.
{1, 2, 3, 4, 5, 6}
.
{0, 2, 4, 6, 8, 10, 12, 14, 16, 18}
.
{1, 2, 3, 4, 5}

```

```

L = list(range(1000000))
S = set(L)

```

```

print(999999 in L)
print(999999 in S)

```

```

True
True

```

Typecasting Various Data Types into Sets

```

Text = "Bangladesh"           # String → Set
S1 = set(Text)
print(S1)

print("\n")

L = [1, 2, 2, 3, 4, 4, 5]      # List → Set
S2 = set(L)
print(S2)

print("\n")

D = {"a": 1, "b": 2, "c": 3}   # Dictionary → Set [Only keys are converted into set.]
S3 = set(D)
print(S3)

print("\n")

S4 = {1, 2, 3}                 # Set → Set (Copying a Set)
S5 = set(S4)                   # Check Whether this is shallow copy or not
print(S5)

```

```

{'h', 'd', 'g', 'n', 'a', 'e', 'B', 'l', 's'}

```

```

{1, 2, 3, 4, 5}

```

```

{'a', 'b', 'c'}

```

```

{1, 2, 3}

```

```

S = set(input("Enter A String : "))

```

```

print(S)

```

```
Enter A String : BANGLADESH
{'L', 'A', 'D', 'N', 'S', 'G', 'B', 'H', 'E'}
```

```
# Sample Example : Student Registration System
```

```
Students = set()
```

```
while True:
```

```
    S_ID = input("Enter Student ID (or Q to Quit): ")
```

```
    if S_ID == 'Q':
```

```
        break
```

```
    Students.add(S_ID)
```

```
print("Registered Students:", Students)
```

```
Enter Student ID (or Q to Quit): 1711441663
```

```
Enter Student ID (or Q to Quit): 1711441664
```

```
Enter Student ID (or Q to Quit): 1711441665
```

```
Enter Student ID (or Q to Quit): 1711441666
```

```
Enter Student ID (or Q to Quit): 1711441667
```

```
Enter Student ID (or Q to Quit): Q
```

```
Registered Students: {'1711441663', '1711441666', '1711441664', '1711441667', '1711441665'}
```